

Lecture 8 Task — Skills & Connectors

Teach AI Once, Connect It to Your Apps

Objective

The purpose of this assignment is to help students learn two powerful ideas from the **Skills & Connectors** crash course: how to **teach an AI a task once** so it can repeat it your exact way (a **Skill**), and how to **safely connect an AI to your real apps** so it can work with your actual data (a **Connector**).

By the end, you should be able to package a task you keep re-explaining into a reusable Skill, connect an app in a read-only way, combine the two, share or move a Skill so it works elsewhere, and audit a Skill for safety before trusting it.

You will complete **five hands-on projects**. Wherever possible, use your own real tasks and real apps so the result is genuinely useful to you.

About These Projects

These five projects come from the **Skills & Connectors** crash course, part of the **The AI Agent Factory** book. Two quick definitions you will use throughout:

- **Skill:** reusable instructions that teach the AI how to do *one task your specific way*. At its simplest a Skill is just a folder with a text file (**SKILL.md**) that contains a name, a description, and plain-English instructions — no code required to start. The AI reads the description to decide when the Skill is relevant, then follows the full instructions only when needed.
- **Connector:** safe, permission-scoped access that gives the AI “hands” to reach your real apps and data (for example Google Drive, Gmail, or Slack).

One rule applies throughout: only connect what you need, prefer read-only access first, and always check the AI’s output against the real source before you trust it.

Book & Reading (read before starting)

- **Book:** The AI Agent Factory — <https://agentfactory.panaversity.org/>
- **This lecture’s chapter:** Skills & Connectors (crash course) — <https://agentfactory.panaversity.org/docs/skills-connectors-crash-course>

Task 1 — Build Your First Real Skill (Your Own Daily-Life Skill)

Estimated time: about 30–45 minutes.

The Idea

Build a Skill that **helps you in your own daily life**. Think of something you do or explain again and again — and turn it into a reusable Skill so the AI does it your exact way every time, just from a normal sentence. **Be creative and make it personal**. The best Skill here is one you will actually keep using after this assignment.

A good Skill takes a task you repeat, encodes *your* preferred format and rules, triggers from natural phrases (not a special command), and produces a consistent result every time.

Ideas to Spark Your Own (pick your own — do not just copy these)

- A daily to-do or stand-up formatter that turns a messy brain-dump into a clean, prioritized list.
- An email/message reply drafter that writes in your personal tone.
- A meal-plan and grocery-list maker from whatever you have at home.
- A workout or habit log that formats your entries and tracks streaks.
- An “explain this in simple words + 3 takeaways” summarizer for articles you read.
- An expense splitter that formats who-owes-what after a group outing.
- A caption / social-media post generator in your own brand voice.
- A “turn a rough idea into a structured plan” helper for your projects.

Steps

1. Choose your own daily-life task and decide exactly how you want the output to look (the format, the rules, the tone).
2. Use the **skill-creator** skill: describe your task in plain English, let it ask you clarifying questions, and answer them so it can capture your exact way of doing it.
3. Let skill-creator generate the **SKILL.md** (a name, a description, and your numbered instructions).
4. Test it: open a **brand-new chat**, give it a normal request *without naming the skill*, and confirm the Skill fires automatically and produces your exact format.
5. Refine the instructions and re-test until it triggers reliably and the output is consistent.

If You Truly Cannot Think of an Idea (fallback only)

Only if you genuinely cannot come up with your own idea, build the weekly study-notes Skill from the course by pasting this exact prompt into skill-creator:

“Use the skill-creator skill to build me a weekly study-notes skill. Whenever I ask for ‘study notes’ or ‘weekly notes’, take my raw class notes and format them as a heading per topic, a short bold key-terms list, and exactly three review questions at the end. Ask me anything you need, then build it.”

Then test it in a new chat by pasting raw notes after the line “Make study notes from this:” (without naming the skill) and confirm it formats them correctly. **Note:** choosing your own idea will score higher than using this fallback.

Expected Outcome

A working Skill saved as a SKILL.md that triggers automatically from natural language and produces your exact format every time — ideally one you will genuinely keep using.

Task 2 — Connect One App, Read-Only

Estimated time: about 20–30 minutes.

The Idea

Give the AI safe, read-only access to one real app and pull real data — no copy-pasting. The point is to feel how a Connector works and to understand exactly what access you are granting.

Requirements

- An app you can connect (for example Google Drive or Gmail).
- Something real to retrieve (a recent document or a recent email thread).

Steps

1. Connect one app (for example Google Drive or Gmail) using the connector.
2. Ask the AI to find and summarize your most recent Drive document, or summarize a recent email thread.
3. Confirm the summary actually matches the real source.
4. In one sentence, write down what access you granted and whether you would ever need write (not just read) permission.

Expected Outcome

The AI pulls genuine data through the connector, and you can clearly state, in one sentence, what permission you gave it.

Task 3 — Wire Skill + Connector Together

Estimated time: about 45–60 minutes.

The Idea

Combine Task 1 and Task 2: use your Skill *and* a Connector in a single workflow, so one plain-English sentence fetches live data and formats it automatically.

Requirements

- The Skill you built in Task 1 (or the study-notes Skill).
- The app you connected in Task 2, containing real data the Skill can format.

Steps

1. Ask the AI, in one normal sentence, to fetch real data from your connected app and format it with your Skill (for example, “pull this week’s class notes from my Drive and make study notes”).
2. Let the Skill trigger automatically and the Connector retrieve the live data.
3. Spot-check one part of the result against the original source before trusting the whole output.

Expected Outcome

A single plain-English sentence produces a finished, correctly formatted result built from live data — no manual copy-paste.

Task 4 — Make It Portable or Hand It Off

Estimated time: about 30 minutes.

The Idea

Prove your Skill survives beyond one conversation and one platform — that it is a real, reusable asset and not a one-time trick.

Steps (choose one path)

- **Hand it off:** share your Skill with another person and have them use it themselves, with no extra explanation from you, and confirm they get the correct result.
- **Make it portable:** load your Skill into a second surface (for example Cowork or Claude Code) and run it there, confirming it still works.

Expected Outcome

Your Skill produces correct results outside its original chat — either for another person or in a different environment — without you having to re-explain it.

Task 5 — Audit a Skill Before Trusting It

Estimated time: about 15–20 minutes.

The Idea

Build the safety habit of vetting a Skill *before* you enable it — especially Skills made by someone else. You should be able to say what a Skill does and whether it is safe, even without reading code.

Steps

1. Install one Skill from the official skills directory.
2. Ask the AI to explain, in plain English, exactly what the Skill does.
3. Ask it to flag anything sensitive: does it contact any external server, handle credentials, or send your data anywhere unexpected?
4. Decide and write down whether the Skill is safe to enable, and why.

Expected Outcome

A plain-English safety assessment in your own words: what the Skill does, what it touches, and whether you would trust it — no code literacy required.

Submission — One GitHub Repository for All Five Projects

Every student must create **one single GitHub repository** that contains all five projects together. Do not create five separate repositories.

Recommended Repository Structure

- **README.md** (top level) — your name, a short summary of the five projects, and which AI tools/apps you used.
- **task-1-my-skill/** — your **SKILL.md**, a short README explaining the daily-life task it solves, the prompts you used, and screenshots of it triggering automatically.
- **task-2-connect-app/** — a README naming the app you connected, your one-sentence note on the permission granted, and screenshots of the result (with private details blurred).
- **task-3-skill-plus-connector/** — a README describing the combined workflow, the sentence you used, and screenshots of the formatted live result.
- **task-4-portable-handoff/** — a README on whether you handed it off or moved it, and proof (a screenshot or the other person's confirmation) that it worked elsewhere.
- **task-5-skill-audit/** — your written safety assessment of the Skill you audited.

Important — never commit secrets. Connectors use logins and permissions. Do not commit any tokens, passwords, OAuth credentials, or private data exports. Blur or remove private details (email addresses, account numbers, personal content) in any screenshots before you upload them.

Each Task Folder Should Contain

- A short README explaining what you did and how you verified it worked.
- The SKILL.md file(s) for any Skill you built (Tasks 1 and 3).
- The prompts you used (initial prompt and any improved versions).
- Screenshots or a short clip showing the working result (private data hidden).

Deliverables

Each student must submit:

- The link to your single GitHub repository containing all five projects.
- A short report (Markdown in the repo, a PDF, or a Google Doc) covering all five projects.
- Screenshots or short demo clips for each project.

The Report Should Include (for each project)

- Project title and what it does.
- Which AI tool(s) and app(s) you used.
- The prompts you used (and how you refined them).
- How you tested or verified that it worked.
- What worked, what did not, and any problems you faced.
- For Task 1: why you chose your Skill and how it helps your daily life.
- For Task 5: your safety assessment and verdict.

Final Submission Requirements

Submit all materials on the Panaversity Dashboard/Portal:

1. **GitHub Repository Link** — one repo with all five task folders.
2. **Report Document** — explaining each Skill/Connector, the prompts used, how you tested it, problems faced, and final results.
3. **Supporting Material** — screenshots, demo video, Google Docs link, PDF report, or any other useful files.

Submission Instructions

- Upload all deliverables to your Panaversity Dashboard/Portal.
- Make sure your GitHub repository is public (or accessible to the instructor) and all links work.
- Organize the repository with one clearly named folder per task.
- Never commit tokens, passwords, or private data; blur private details in screenshots.
- Make sure each Skill triggers and each Connector retrieves real data before you submit.

Evaluation Criteria

Students will be evaluated on:

- **Creativity and usefulness of the Task 1 Skill** — an original, genuinely helpful daily-life Skill scores higher than the fallback example.
- Whether the Skill triggers automatically from natural language and gives a consistent format.
- Correct, safe use of a Connector to retrieve real data (read-only first).
- Successfully wiring Skill + Connector into one working workflow.
- Proving the Skill is portable or can be handed off to someone else.
- Quality of the Task 5 safety audit and understanding of permissions.
- Quality of documentation, prompts, and the written report.
- Completeness — all five tasks attempted and organized in one repository.